Solution: Best Time to Buy and Sell Stock II

Problem Statement

You are given a `prices` array containing the integers, where `price[i]` refers to the stock price on i^{th} day.

You are allowed to do as many transactions as you like (buy one and sell one stock multiple times). You cannot engage in multiple transactions `simultaneously` (you must sell the stock before you buy again). However, you can buy again on the `same day` after selling.

Find the `maximum profit` you can achieve from these transactions.

Examples

- **Example 1**:
  - **Input**: `[4, 9, 2, 3, 6, 8]`
  - **Expected Output**: `11`
  - **Justification**: Buy on day 1 (price = 4) and sell on day 2 (price = 9), profit = 9-4 = 5. Then buy on day 3 (price = 2) and sell on day 6 (price = 8), profit = 8-2 = 6. Total profit is 5 + 6 = 11.
- **Example 2**:
  - **Input**: `[5, 3, 6, 2, 5, 4]`
  - **Expected Output**: `6`
  - **Justification**: Buy on day 2 (price = 3) and sell on day 3 (price = 6), profit = 6-3 = 3. Then buy on day 4 (price = 2) and sell on day 5 (price = 5), profit = 5-2 = 3. Total profit is 3 + 3 = 6.
- **Example 3**:
  - **Input**: `[10, 8, 6, 5, 4, 2]`
  - **Expected Output**: `0`
  - **Justification**: Prices are decreasing each day, so there's no opportunity to make a profit. It is best not to make any transactions.

Solution

To solve this problem, we'll adopt a greedy approach, focusing on capturing every possible profitable transaction. The key idea is to accumulate profit from every upward trend, which is identified by buying at the start of the trend (local minimum) and selling at the end (local maximum). This strategy ensures we don't miss any profit opportunity by trying to predict the highest peak or lowest valley ahead of time.

This approach is effective because it capitalizes on the fact that the sum of profits from each consecutive transaction (buy low, sell high) adds up to the maximum possible profit. It aligns with the principle of taking advantage of every positive price difference between consecutive days. By summing up all positive differences, we can maximize the profit without needing to know the overall price pattern in advance.

Step-by-step Algorithm

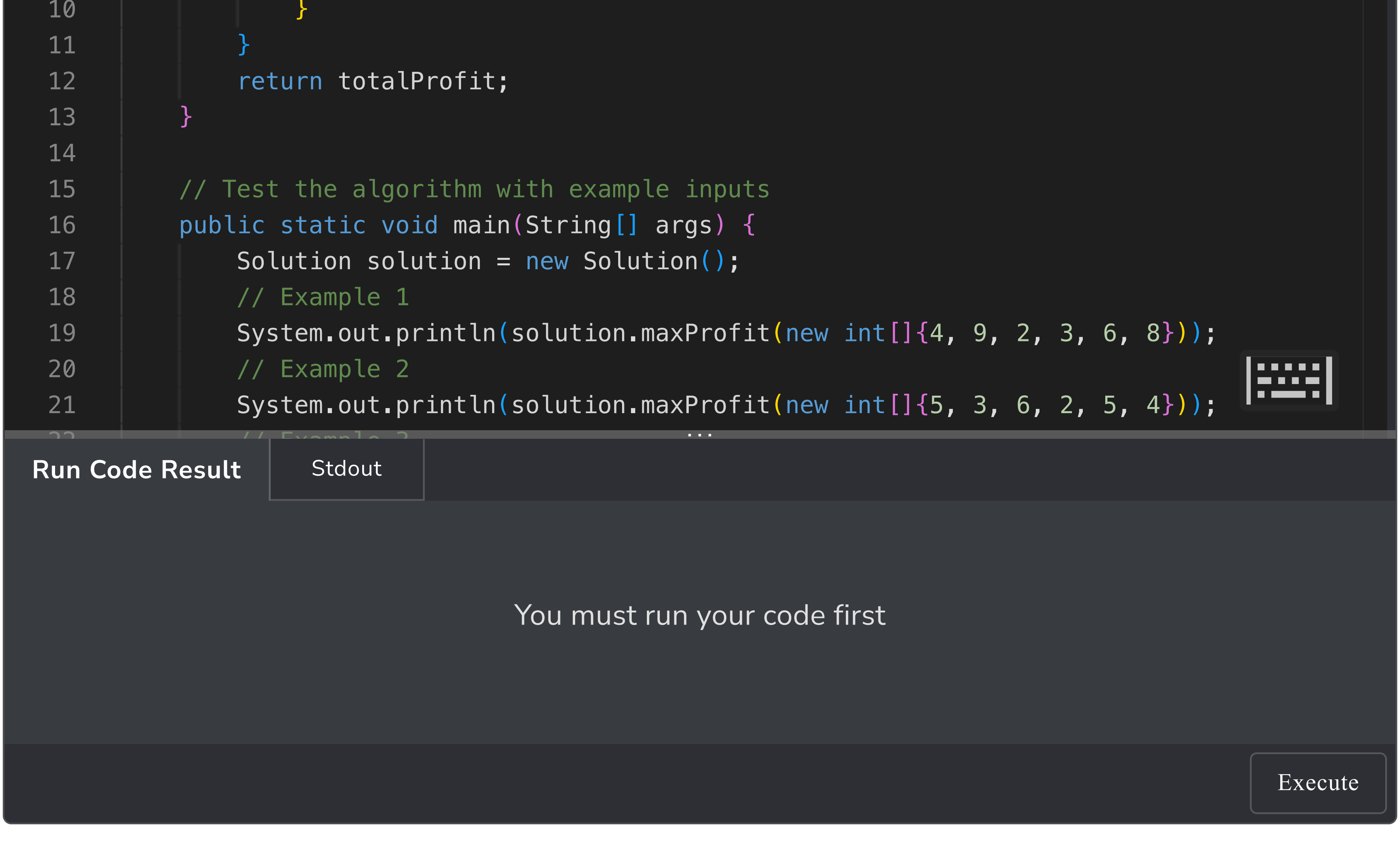
- Initialize `totalProfit` to 0.
- Iterate through the price array from the first day to the last day.
  - For each day, compare the current day's price to the previous day's price.
  - If the current day's price is higher than the previous day's price:
    - Calculate the difference (profit) and add it to `totalProfit`.
- Return `totalProfit` as the maximum profit that can be achieved.

5. Algorithm Walkthrough

Let's consider the input `[5, 3, 6, 2, 5, 4]`:

1. **Initialization**:
   - Start with `totalProfit = 0`.
   - The input prices are `[5, 3, 6, 2, 5, 4]`.
2. **Day 1 to Day 2**:
   - Prices move from 5 to 3. There's no profit since the price decreased, so we do nothing.
   - `totalProfit` remains `0`.
3. **Day 2 to Day 3**:
   - Prices move from 3 to 6. This is a profitable move.
   - Buy at 3 and sell at 6, making a profit of `6 - 3 = 3`.
   - Update `totalProfit` to `3`.
4. **Day 3 to Day 4**:
   - Prices move from 6 to 2. Price decreased, so we do nothing.
   - `totalProfit` is still `3`.
5. **Day 4 to Day 5**:
   - Prices move from 2 to 5. Another opportunity to profit.
   - Buy at 2 and sell at 5, making a profit of `5 - 2 = 3`.
   - Update `totalProfit` to `3 + 3 = 6`.
6. **Day 5 to Day 6**:
   - Prices move from 5 to 4. Price decreased, so we do nothing.
   - `totalProfit` remains `6`.
7. **Conclusion**:
   - By the end of the last trading day, we've captured all profitable transactions by buying low and selling high.
   - The final `totalProfit` is `6`.

Code



```java
public class Solution {
    // Calculate the maximum profit from stock transactions
    public int maxProfit(int[] prices) {
        int totalProfit = 0;
        // Iterate over array to find successive profit opportunities
        for (int i = 1; i < prices.length; i++) {
            // Add to total profit if today's price is higher than yesterday's
            if (prices[i] > prices[i - 1]) {
                totalProfit += prices[i] - prices[i - 1];
            }
        }
        return totalProfit;
    }

    // Test the algorithm with example inputs
    public static void main(String[] args) {
        Solution solution = new Solution();
        // Example 1
        System.out.println(solution.maxProfit(new int[]{4, 9, 2, 3, 6, 8}));
        // Example 2
        System.out.println(solution.maxProfit(new int[]{5, 3, 6, 2, 5, 4}));
```

You must run your code first

Complexity Analysis

Time Complexity

- $O(n)$: The algorithm iterates through the array of stock prices exactly once. Here, `n` represents the number of days for which we have stock prices.

Space Complexity

- $O(1)$: The algorithm uses a fixed amount of extra space to store variables (`totalProfit` and loop index `i`). This usage does not scale with the input size, making the space complexity constant.

← Previous
Best Time To Buy And Sell Stock II (Medium)

Next →
Largest Palindromic Number (Medium)

☐ Mark as Completed